Machine learning

Pr. OMARI Kamal

FACULTE POLYDISCIPLINAIRE D'OUARZAZATE

16 décembre 2024

Chapitre 2 : Le Prétraitement des Données en Machine Learning

1. Introduction
   - Introduction
   - Types de données
2. Étapes du Prétraitement
   - Compréhension des Données
   - Nettoyage des Données
   - Transformation des Données
   - Normalisation et standardisation
   - La réduction de la dimensionnalité
   - Gestion des Données Déséquilibrées
3. Conclusion

Objectifs

Dans ce chapitre, nous allons :

- Comprendre les différents types de données (numériques, catégorielles, etc.) et leur traitement adapté en Machine Learning.
- Apprendre les étapes de nettoyage des données pour identifier et traiter les valeurs manquantes, les anomalies et les erreurs.
- Explorer les techniques de transformation des données, telles que la normalisation, la standardisation, et la réduction de la dimensionnalité.
- Gérer les données déséquilibrées pour améliorer la performance des modèles en équilibrant les classes.

Introduction

Le prétraitement des données est l'ensemble des étapes permettant de convertir les données brutes en un format optimal pour les algorithmes de Machine Learning. Ces étapes incluent le nettoyage, la transformation, la réduction de la dimensionnalité, et la préparation des ensembles de données pour l'entraînement.

Pourquoi le Prétraitement est-il Nécessaire ?

- **Qualité des données :** Les données collectées sont souvent bruyantes, incomplètes ou incohérentes.
- **Compatibilité :** Certains algorithmes (e.g., SVM, régressions) nécessitent des données normalisées ou numériques.
- **Efficacité :** Des données bien prétraitées réduisent le temps de calcul et améliorent la précision des modèles.
- **Atténuation des déséquilibres :** Le prétraitement permet de réduire les déséquilibres présents dans la distribution des données.

Types de données

Dans le contexte du Machine Learning, les données peuvent être classées en plusieurs types, chacun ayant des caractéristiques et des méthodes spécifiques de traitement et d'analyse. Ces types de données sont fondamentaux pour choisir les algorithmes et les techniques appropriées à appliquer.

Données Numériques

Les données numériques sont des valeurs qui peuvent être mesurées et qui ont une signification quantitative. Elles sont divisées en deux catégories principales :

Données continues

Définition : Les données continues peuvent prendre n'importe quelle valeur dans un intervalle donné, avec une précision infinie.
Exemples :

- Température (en °C ou °F)
- Taille d'une personne (en mètres ou centimètres)
- Poids d'un objet (en kilogrammes)
- Vitesse d'un véhicule (en km/h)

Traitement : Ces données peuvent être utilisées directement dans la plupart des modèles de Machine Learning après **normalisation** ou **standardisation**, pour garantir qu'elles aient la même échelle.

Données Discrètes

Définition : Les données discrètes prennent un nombre limité de valeurs distinctes. Elles sont généralement comptées et non mesurées.

Exemples :

- Nombre d'enfants dans une famille
- Nombre d'unités vendues
- Nombre d'incidents dans une journée

Traitement : Ces données sont souvent traitées sous forme d'entiers et peuvent nécessiter un encodage, comme l'encodage **one-hot**, lorsqu'elles sont utilisées dans des modèles comme les réseaux neuronaux.

Données Catégorielles

Les données catégorielles représentent des qualités ou des catégories. Elles ne peuvent pas être mesurées numériquement, mais sont utilisées pour représenter des attributs ou des groupes.

Données Nominales

Définition : Les données nominales sont des catégories qui n'ont pas d'ordre naturel ou de hiérarchie.

Exemples :

- Couleurs (rouge, bleu, vert)
- Marque d'un produit (Apple, Samsung, Sony)
- Types de fruits (pomme, banane, orange)

Traitement : L'encodage des données nominales est généralement effectué avec des techniques comme l'encodage **one-hot**, qui transforme chaque catégorie en une colonne distincte.

Données Ordinales

Définition : Les données ordinales représentent des catégories qui ont un ordre ou un classement.

Exemples :

- Évaluation d'un produit (très mauvais, mauvais, moyen, bon, excellent)
- Niveaux de satisfaction (insatisfait, neutre, satisfait)
- Grades scolaires (A, B, C, D, F)

Traitement : Les données ordinales peuvent être encodées avec un encodage numérique qui respecte l'ordre (par exemple, 1 pour "mauvais", 5 pour "excellent"). Cependant, dans certains cas, l'encodage **one-hot** peut aussi être utilisé si l'on ne veut pas attribuer de valeur numérique.

Compréhension des Données

Avant de construire un modèle ou d'effectuer une analyse, il est essentiel de comprendre et d'explorer les données. Cette étape garantit une meilleure prise de décision et des résultats fiables.

Inspection Initiale

L'exploration des données commence par leur chargement et une première inspection pour comprendre leur structure et leur contenu.

- **Chargement des données :** Utilisez des bibliothèques comme **pandas** pour charger les fichiers courants tels que .csv, .xlsx, ou bases de données SQL.

Exemple

```
import pandas as pd
# Charger un fichier CSV
df = pd.read_csv('nom_du_fichier.csv')
```

Examen rapide des données

Afficher les premières et dernières lignes : Cela permet de visualiser la structure et le contenu du jeu de données.

Exemple

```
df.head() # Affiche les 5 premières lignes
df.tail() # Affiche les 5 dernières lignes
```

Examen rapide des données

Dimensions des données : Vérifiez la taille pour connaître le nombre de lignes et de colonnes.

Exemple

```
print(f'Dimensions : {df.shape}')
```

Résumé Statistique

Un résumé statistique permet de mieux comprendre la répartition et les caractéristiques des données.

- Informations générales sur le dataset
  - info() affiche le type de données, le nombre de valeurs non nulles et la mémoire utilisée.
  - Cela est utile pour repérer des colonnes avec des valeurs manquantes.

Exemple

```
df.info()
```

Résumé Statistique

- Statistiques descriptives : Utilisez describe() pour obtenir des statistiques clés comme la moyenne, l'écart type, les valeurs minimales et maximales.

Exemple

```
df.describe() # Résumé statistique pour les colonnes
numériques
```

Visualisation des Données

La visualisation est essentielle pour détecter les tendances, les corrélations et les anomalies.

- Bibliothèques courantes :
  - Matplotlib : Idéal pour les visualisations simples.
  - Seaborn : Fournit des graphiques élégants et riches en fonctionnalités.

Exemples de visualisations :

- Histogramme : Pour observer la distribution des données.

Exemple

```
import matplotlib.pyplot as plt
df['colonne_interessante'].hist(bins=20, color='blue')
plt.title('Distribution des données')
plt.show()
```

Visualisation des Données

Heatmap de corrélation : Pour identifier les relations entre les variables.

Exemple

```
import seaborn as sns
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Matrice de corrélation')
plt.show()
```

Nettoyage des Données

Le nettoyage des données est une étape cruciale dans la préparation des données avant leur analyse. Il vise à éliminer les erreurs, incohérences et valeurs manquantes, tout en assurant une qualité optimale des données pour les modèles de machine learning.

Traitement des valeurs manquantes

Les valeurs manquantes dans un jeu de données peuvent être traitées de différentes manières, en fonction de leur distribution et de leur impact sur l'analyse.

Repérer les valeurs nulles ou manquantes

Lors de l'analyse de données, il est important de vérifier la présence de valeurs nulles (ou valeurs manquantes). Ces valeurs peuvent impacter les résultats de vos analyses ou modèles et doivent être traitées.

Pourquoi repérer les valeurs nulles ?

- Les valeurs nulles indiquent l'absence d'une donnée dans un ensemble.
- Leur présence peut causer des erreurs ou fausser les calculs dans des analyses statistiques ou algorithmes de machine learning.
- Identifier ces valeurs permet de décider comment les gérer (remplissage, suppression, etc.).

Repérer les valeurs nulles ou manquantes

Pandas propose la méthode isnull() qui permet de détecter les valeurs nulles dans un DataFrame ou une série. Associée à la fonction sum(), elle permet de compter les valeurs nulles pour chaque colonne.

Exemple

```
print(data.isnull().sum())
```

Supprimer les lignes ou colonnes contenant des valeurs nulles

Une première approche consiste à supprimer les lignes ou les colonnes qui contiennent des valeurs nulles, mais cette méthode est plus appropriée si ces valeurs manquantes sont rares.

Code pour supprimer les lignes contenant des valeurs nulles

```
df.dropna(inplace=True)
```

- dropna() supprime par défaut toutes les lignes où une ou plusieurs valeurs sont nulles.
- L'option inplace=True permet de modifier le DataFrame directement, sans créer une copie.

Supprimer les lignes ou colonnes contenant des valeurs nulles

Code pour supprimer les colonnes contenant des valeurs manquantes

```
df.dropna(axis=1, inplace=True)
```

- axis=1 indique que la suppression concerne les colonnes (par défaut, axis=0 supprime les lignes).

Imputer les valeurs manquantes

L'imputation consiste à remplacer les valeurs manquantes par une valeur estimée, comme la moyenne ou la médiane pour les variables numériques, ou la valeur la plus fréquente pour les variables catégoriques.

Imputation avec la moyenne pour les variables numériques

```
from sklearn.impute import SimpleImputer
# Imputation des valeurs manquantes avec la moyenne
imputer = SimpleImputer(strategy='mean')
df['colonne'] = imputer.fit_transform(df[['colonne']])
```

- SimpleImputer(strategy='mean') remplace les valeurs manquantes par la moyenne des valeurs présentes dans la colonne spécifiée.
- fit_transform() ajuste l'imputer et applique l'imputation sur la colonne.

Imputer les valeurs manquantes

Imputation avec la valeur la plus fréquente pour les variables catégoriques

```
# Imputation des valeurs manquantes avec la valeur la plus fréquente
imputer = SimpleImputer(strategy='most_frequent')
df['colonne_categorique'] = imputer.fit_transform(df[['colonne_categorique']])
```

- strategy='most_frequent' remplace les valeurs manquantes par la valeur la plus fréquente dans la colonne catégorique.

Gestion des doublons

Les doublons peuvent fausser l'analyse en influençant les résultats de manière incorrecte. Il est donc important de les identifier et de les supprimer.
Voici comment procéder pour les supprimer :

Suppression des doublons

```
df.drop_duplicates(inplace=True)
```

- drop_duplicates() supprime toutes les lignes identiques dans le DataFrame.

Correction des valeurs aberrantes

Les valeurs aberrantes peuvent influencer les résultats des analyses statistiques et des modèles d'apprentissage automatique. Il existe diverses techniques pour les identifier et les gérer.

Utilisation de l'IQR (Plage Interquartile)

L'IQR est une méthode fréquemment utilisée pour détecter les valeurs aberrantes. Elle se base sur la distribution des données et permet d'identifier les valeurs qui s'écartent excessivement des quartiles.

Calcul de l'IQR et identification des outliers

```
# Calcul de l'IQR
Q1 = df['colonne'].quantile(0.25)
Q3 = df['colonne'].quantile(0.75)
IQR = Q3 - Q1
# Définition des bornes pour les outliers
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
# Filtrer les valeurs aberrantes
df_cleaned = df[(df['colonne'] >= lower_bound) & (df['colonne'] <= upper_bound)]
```

Exemple

Exemple

```
import pandas as pd
# Données
data = [20, 25, 30, 35, 110, 50, 55, 60, 100, 45]
# Créer un DataFrame
df = pd.DataFrame(data, columns=["Age"])
# Calcul des quartiles
Q1 = df["Age"].quantile(0.25)
Q3 = df["Age"].quantile(0.75)
# Calcul de l'IQR
IQR = Q3 - Q1
# Calcul des bornes pour détecter les outliers
borne_inf = Q1 - 1.5 * IQR
borne_sup = Q3 + 1.5 * IQR
# Identification des outliers
outliers = df[(df["Age"] < borne_inf) | (df["Age"] > borne_sup)]
# Affichage des résultats
print(f"Q1: {Q1}et Q3: {Q3}")
print(f"IQR: {IQR}")
print(f"Borne inférieure: {borne_inf}")
print(f"Borne supérieure: {borne_sup}")
print("Outliers:")
print(outliers)
```

En résumé

Le nettoyage des données est une étape fondamentale dans l'analyse de données, car il permet d'améliorer la qualité des données avant de les utiliser dans des modèles. Le traitement des valeurs manquantes, la gestion des doublons et la correction des valeurs aberrantes sont trois étapes essentielles pour garantir des résultats fiables et précis. Ces techniques doivent être adaptées en fonction de la nature des données et des objectifs de l'analyse.

Transformation des Données

La transformation des données est une étape cruciale en science des données. Elle permet de préparer les données brutes pour les rendre exploitables par des modèles d'apprentissage. Voici les techniques les plus courantes.

Encodage des variables catégoriques

Les variables catégoriques doivent être converties en valeurs numériques pour être utilisées par les algorithmes de machine learning. Il existe deux principales approches :

Encodage ordinal

Encodage ordinal :
Lorsque les catégories possèdent un ordre logique (par exemple : petite, moyenne, grande), on leur attribue des valeurs numériques reflétant cet ordre.

Encodage ordinal

```
df['taille'] = df['taille'].map('petite': 1,
'moyenne': 2, 'grande': 3)
```

Avantages et Inconvénients

Avantages :

- Simple et efficace pour les données ordinales.
- Conserve l'ordre des catégories.

Inconvénients :

- Ne convient pas pour des catégories sans relation d'ordre.

Encodage one-hot

L'encodage **one-hot** est une technique couramment utilisée en apprentissage automatique pour convertir des variables catégoriques (nominales) en une représentation numérique compréhensible pour les modèles.

Principe de l'encodage one-hot

- Pour chaque catégorie unique d'une variable, on crée une nouvelle colonne (ou un nouveau "feature").
- Chaque observation (ligne) sera représentée par une combinaison de valeurs binaires 1 ou 0, indiquant la présence ou l'absence de chaque catégorie.

Exemple pas à pas

Données initiales :
Supposons une variable catégorique `categorie` avec les valeurs suivantes dans un DataFrame :

categorie
A
B
C

Résultat après encodage one-hot :
Après l'encodage one-hot, chaque catégorie aura sa propre colonne, et les données seront transformées comme suit :

[A]	**[B]**	**[C]**
1	0	0
0	1	0
0	0	1

Cela signifie :

- Si la catégorie est `"A"`, la colonne `[A]` sera 1, et les autres colonnes `[B]` et `[C]` seront 0.
- Pareillement pour `"B"` et `"C"`.

Mise en œuvre avec Python

Voici le code pour encoder ces données à l'aide de la bibliothèque scikit-learn :

Encodage ordinal

```
from sklearn.preprocessing import OneHotEncoder
import pandas as pd
# Exemple de DataFrame
df = pd.DataFrame('categorie': ['A', 'B', 'C'])
# Initialisation de l'encodeur
encoder = OneHotEncoder()
# Encodage des données
encoded = encoder.fit_transform(df[['categorie']]).toarray()
# Afficher le résultat
print(encoded)
```

Avantages de l'encodage One-Hot

Convient aux catégories nominales :

- Les catégories nominales n'ont pas d'ordre logique ou de hiérarchie. Par exemple, `"rouge"`, `"vert"`, `"bleu"` sont juste des labels.
- L'encodage one-hot permet de représenter ces catégories sans introduire d'ordre implicite (comme le ferait un encodage par des chiffres : 1, 2, 3).

Évite les hypothèses erronées des modèles :

- Si l'on encode `"A"`, `"B"`, `"C"` avec des nombres (par exemple, 1, 2, 3), certains algorithmes pourraient penser que `"C"` (3) est plus proche de `"B"` (2) que de `"A"` (1), ce qui n'a pas de sens pour des catégories nominales.

Inconvénients de l'encodage One-Hot

Augmentation de la dimensionnalité :

- Si une variable a beaucoup de catégories uniques (par exemple, des codes postaux ou des ID de produits), l'encodage one-hot peut générer un très grand nombre de colonnes, augmentant la taille de la matrice.
- Cela peut poser des problèmes de mémoire et ralentir les algorithmes d'apprentissage.

Non optimal pour certaines catégories rares :

- Si certaines catégories apparaissent rarement, leur colonne correspondante sera majoritairement composée de 0 (*sparse data*), ce qui peut ne pas être idéal pour certains modèles.

En résumé

En résumé, l'encodage one-hot est simple et efficace pour traiter des variables catégoriques nominales, mais il peut devenir coûteux en termes de ressources avec des ensembles de données contenant de nombreuses catégories.

Normalisation et standardisation

Ces deux techniques sont utilisées pour transformer les données numériques et rendre leurs échelles comparables. Elles sont souvent essentielles pour les algorithmes qui sont sensibles aux échelles ou aux magnitudes des données.

Pourquoi uniformiser les échelles ?

Certains algorithmes, comme K-Nearest Neighbors (KNN), Support Vector Machine (SVM), ou réseaux neuronaux, sont influencés par les différences d'échelle entre les variables. Si une variable a des valeurs beaucoup plus grandes que les autres, elle peut dominer les calculs de distance ou d'optimisation.

Normalisation

La normalisation met toutes les valeurs dans une plage fixe, généralement entre 0 et 1.
Elle est basée sur les valeurs minimales et maximales des données.
La formule est :

$$\text{Valeur normalisée} = \frac{\text{Valeur} - \text{Min}}{\text{Max} - \text{Min}}$$

Cas d'utilisation :

- Très utile pour les algorithmes basés sur les distances (KNN, K-means), car ils utilisent des distances euclidiennes où les magnitudes ont un impact direct.

Normalisation

Exemple en Python

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd
# Exemple de données
data = {'col1': [10, 20, 30, 40, 50], 'col2': [200,
300, 400, 500, 600]}
df = pd.DataFrame(data)
# Normalisation
scaler = MinMaxScaler()
df_scaled = scaler.fit_transform(df)
# Affichage du résultat
print(df_scaled)
```

Standardisation

Principe :
La standardisation transforme les données pour qu'elles aient une moyenne de 0 et un écart-type de 1.
Cela est utile lorsque les données doivent suivre une distribution normale (gaussienne).
La formule est :

$$\text{Valeur standardisée} = \frac{\text{Valeur} - \text{Moyenne}}{\text{Écart-type}}$$

Cas d'utilisation :

- Très utile pour les algorithmes comme les *Support Vector Machines* (SVM), les réseaux neuronaux, et la régression linéaire/logistique.
- Ces algorithmes supposent souvent que les données ont une distribution normale ou symétrique.

Standardisation

Exemple en Python

```
from sklearn.preprocessing import StandardScaler
# Standardisation
scaler = StandardScaler()
df_standardized = scaler.fit_transform(df)
# Affichage du résultat
print(df_standardized)
```

Comparaison : Normalisation vs Standardisation

Aspect	**Normalisation**	**Standardisation**
Objectif	Uniformiser la plage des valeurs	Uniformiser la distribution des données
Méthode	Basé sur le min et le max des données	Basé sur la moyenne et l'écart-type
Plage des valeurs	Généralement entre 0 et 1	Moyenne 0, écart-type 1
Cas d'utilisation	Algorithmes basés sur les distances	Algorithmes sensibles aux distributions
Exemples d'algorithmes	KNN, K-means, PCA	SVM, Régression linéaire, Réseaux neuronaux

En Résumé

- Normalisation : Échelle entre 0 et 1, utile pour les distances.
- Standardisation : Moyenne 0 et écart-type 1, utile pour la normalisation des distributions.

En fonction de l'algorithme et de vos données, choisissez la technique qui maximise les performances du modèle.

La réduction de la dimensionnalité

La réduction de la dimensionnalité est une technique importante en apprentissage automatique qui permet de simplifier les données tout en conservant l'essentiel de l'information. Cela aide à réduire la complexité computationnelle et à améliorer la performance des modèles.

Pourquoi réduire la dimensionnalité ?

- Améliorer la performance des modèles : Trop de variables inutiles ou redondantes peuvent nuire à la précision.
- Réduire le surapprentissage : Trop de caractéristiques peuvent entraîner un modèle qui sur-apprend les données d'entraînement mais échoue sur de nouvelles données.
- Accélérer les calculs : Réduire la dimensionnalité diminue le temps de calcul pour les algorithmes lourds.
- Faciliter la visualisation : Les données de grande dimension sont difficiles à interpréter. La réduction peut aider à visualiser les données en 2D ou 3D.

Analyse de la pertinence des variables

Suppression des variables inutiles ou fortement corrélées

- Lorsque deux variables sont fortement corrélées, l'une peut être supprimée sans perte significative d'information.
- Une matrice de corrélation permet d'évaluer ces relations.

Suppression des variables inutiles ou fortement corrélées

Exemple en Python

```
import seaborn as sns
import pandas as pd
# Exemple de données
data = {'var1': [1, 2, 3, 4, 5], 'var2': [5, 4, 3, 2,
1], 'var3': [1, 3, 5, 7, 9]}
df = pd.DataFrame(data)
# Affichage de la matrice de corrélation
sns.heatmap(df.corr(), annot=True)
```

Sortie possible : Une heatmap qui montre les coefficients de corrélation entre les variables. Les valeurs proches de 1 ou -1 indiquent une forte corrélation.

PCA (Principal Component Analysis)

La PCA projette les données dans un nouvel espace où les dimensions principales (composantes principales) capturent le maximum de variance des données. Cela permet de réduire la dimension tout en conservant l'essentiel de l'information.

PCA (Principal Component Analysis)

Exemple en Python

```
from sklearn.decomposition import PCA
import numpy as np
# Exemple de données
data = np.random.rand(100, 5) # 100 échantillons avec 5 caractéristiques
# PCA pour réduire à 2 dimensions
pca = PCA(n_components=2)
df_pca = pca.fit_transform(data)
print("Données réduites :", df_pca[:5]) # Affiche les 5 premières lignes
des données réduites
```

- n_components = 2 : Réduit les données à 2 dimensions.
- Cela est utile pour des visualisations ou pour réduire les dimensions avant de passer les données à un modèle.

Gestion des Données Déséquilibrées

Les jeux de données déséquilibrés sont courants dans de nombreuses applications de Machine Learning, notamment pour la détection de fraudes, le diagnostic médical, et la détection de courriels de phishing. Un dataset est dit déséquilibré lorsque les classes cibles ne sont pas réparties uniformément (e.g., 90% de classe A et 10% de classe B). Cela peut entraîner des performances biaisées des modèles, car ils tendent à privilégier la classe majoritaire.

Problèmes Posés par les Données Déséquilibrées

- Faible rappel pour la classe minoritaire : Le modèle ignore ou malclasse souvent la classe minoritaire.
- Précision trompeuse : Une haute précision peut masquer le fait que la classe minoritaire n'est pas bien prédite.
- Impact sur les métriques : Les métriques comme l'accuracy ne reflètent pas bien les performances globales.

Pourquoi gérer les données déséquilibrées ?

- Problème courant : Les algorithmes de machine learning ont tendance à privilégier la classe majoritaire lorsqu'il y a un déséquilibre, car ils optimisent souvent la précision globale.
- Impact négatif : Cela peut entraîner de faibles performances sur la classe minoritaire, qui est souvent la classe d'intérêt (ex. détection de fraudes, maladies rares, etc.).

Oversampling (Suréchantillonnage)

Oversampling (Suréchantillonnage) Objectif :

- Augmenter artificiellement la proportion de la classe minoritaire en générant de nouveaux exemples.

Technique courante :
SMOTE (Synthetic Minority Oversampling Technique)

- Génère des exemples synthétiques pour la classe minoritaire en interpolant entre les points existants.

Avantage :

- Évite les doublons créés par un simple clonage.

Limite :

- Peut introduire du bruit si les données minoritaires sont trop dispersées.

Oversampling (Suréchantillonnage)

Exemple avec SMOTE (Python)

```
from imblearn.over_sampling import SMOTE
# Application de SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)
print("Distribution après suréchantillonnage :",
y_resampled.value_counts())
```

Undersampling (Sous-échantillonnage)

Undersampling (Sous-échantillonnage) Objectif :

- Réduire le nombre d'exemples de la classe majoritaire pour équilibrer les classes.

Technique courante :
RandomUnderSampler

- Supprime aléatoirement des exemples de la classe majoritaire.

Avantage :

- Réduit la taille du jeu de données, ce qui accélère l'entraînement.

Limite :

- Risque de perte d'informations importantes.

Undersampling (Sous-échantillonnage)

Exemple avec RandomUnderSampler (Python

```
from imblearn.under_sampling import RandomUnderSampler
# Application de l'undersampling
rus = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = rus.fit_resample(X, y)
print("Distribution après sous-échantillonnage :",
y_resampled.value_counts())
```

Comparaison : Oversampling vs Undersampling

Technique	Avantages	Inconvénients
Oversampling	• Préserve tous les exemples existants • Améliore la performance sur la classe minoritaire	• Peut introduire des exemples synthétiques non réalistes • Augmente la taille du jeu de données
Undersampling	• Réduit la taille du jeu de données (gain de temps) • Simplicité	• Risque de perte d'information importante

Conclusion

En conclusion, le prétraitement des données est une étape fondamentale pour garantir la qualité des modèles de Machine Learning. Il permet de transformer des données brutes en un format adapté, en les nettoyant, les normalisant et les équilibrant si nécessaire. Une bonne gestion des données déséquilibrées et des transformations appropriées peut considérablement améliorer les performances des modèles. Ainsi, maîtriser ces techniques est essentiel pour obtenir des résultats fiables et efficaces en Machine Learning.

QUIZ

Scannez le code QR et répondez aux questions.

Figure 1 – QUIZ